



DELHI TECHNICAL CAMPUS
Greater Noida
Affiliated to GGSIPU and Approved by AICTE & COA



Project Based Learning Report
On
Credit Card Loan Approval
BACHELOR OF TECHNOLOGY
In
COMPUTER SCIENCE & ENGINEERING

SUBMITTED BY

CSE IIIrd Year Section-A

Abhinav Chauhan(00918002723)

GUIDED BY

Dr. Monalisa Panigrahi

(Assistant Professor)



Affiliated to GGSIP University, New Delhi
Approved by AICTE & Council of Architecture

Department of Computer Science & Engineering
DELHI TECHNICAL CAMPUS, GREATER NOIDA
(Affiliated by Guru Gobind Singh Indraprastha University, New Delhi)
Session 2025-26 (Odd Sem)

Introduction:

The financial services industry is currently undergoing a paradigm shift driven by Artificial Intelligence. Traditional credit scoring models, while reliable, often lack the flexibility to adapt to rapidly changing economic conditions and complex regulatory frameworks. The **Multi-Agent Credit Risk & Loan Approval System** is a next-generation solution designed to bridge the gap between automated efficiency and human-like reasoning.

By leveraging a **Multi-Agent System (MAS)** architecture, this project moves away from monolithic, linear processing. Instead, it employs a team of specialized AI agents—powered by Large Language Models (LLMs)—that collaborate to evaluate a single loan application. Each agent acts as a subject matter expert, focusing on data integrity, regulatory compliance, or financial risk synthesis.

The integration of **Retrieval-Augmented Generation (RAG)** allows the system to remain dynamic. Instead of hard-coding lending rules, the system "reads" a living document—the `lending_policy.txt`—ensuring that every decision is grounded in the most current 2026 regulatory standards. This approach provides a level of transparency and auditability that was previously unattainable in fully automated systems.

this project addresses the growing demand for **Explainable AI (XAI)**. In today's regulatory climate, "the AI said so" is no longer an acceptable justification for denying a loan. Customers and regulators alike demand transparency. This system is designed to provide a "Reasoning Trace"—a step-by-step log of the internal dialogue between agents. This trace explains the transition from raw data to the final decision, highlighting which specific policies were met and where the risks were identified. By combining the power of Large Language Models (LLMs) like GPT-4o with structured Pythonic tools, we create a system that is not only highly efficient but also deeply accountable and auditable, setting a new standard for trust in automated financial technology.

Objective :

The primary objective of the **Multi-Agent Credit Risk & Loan Approval System** is to engineer an autonomous, end-to-end decision engine that replicates the sophisticated judgment of a human credit underwriter while maintaining the speed and scalability of a digital platform. This project is not merely about calculating a score; it is about building a system that can reason through financial data, justify its conclusions, and remain compliant with the intricate legal frameworks of 2026. The objectives are categorized into four key pillars: architectural innovation, regulatory agility, technical integration, and user-centric transparency.

Architectural Innovation through Multi-Agent Workflows The first pillar involves implementing a sophisticated orchestration layer using **LangGraph** or **CrewAI**. The objective is to move away from "flat" prompt engineering. By creating specialized roles—the Data Auditor, the Policy Officer, and the Risk Orchestrator—we ensure that each facet of the loan application is scrutinized by a dedicated logic gate. The Auditor ensures data integrity, the Officer ensures legal adherence, and the Orchestrator synthesizes these perspectives into a final verdict. This modularity allows for easier debugging, testing, and future scaling.

Regulatory Agility via Retrieval-Augmented Generation (RAG) A major goal is to solve the problem of "hard-coded" logic. Financial regulations are volatile, especially in the wake of the 2026 fiscal reforms. The objective is to utilize **ChromaDB** or **FAISS** to create a semantic index of internal lending policies. This enables the "Policy Officer" agent to perform high-speed similarity searches, retrieving the most relevant clauses for a specific applicant's profile (eg, specific rules for self-employed individuals vs. salaried employees). This makes the system "future-proof," as it can be updated with new documents without needing to refactor the core codebase.

Technical Integration and Tool-Augmented Reasoning To simulate a real-world production environment, the project aims to integrate external data sources via Pythonic tools. The objective is to define and implement tools like `fetch_credit_report` and `bank_internal_check`. These tools allow the LLM to step out of its "reasoning bubble" and interact with deterministic databases. This hybrid approach ensures that the agents have access to factual, real-time data—such as a user's current credit score and account history—rather than relying solely on the information provided in the input form.

The Problem:

The traditional loan approval process is currently plagued by a trifecta of inefficiencies: human subjectivity, technical obsolescence, and a lack of transparency. These issues lead to increased operational costs, higher default rates, and significant regulatory risks. In the 2026 financial landscape, where data moves at the speed of light and regulations change quarterly, these old-fashioned methods are no longer sustainable.

The Failure of Manual Underwriting and Subjectivity Manual loan processing is fundamentally unscalable. Human underwriters are limited by their cognitive load and are susceptible to unconscious biases. Factors like the time of day, fatigue, or personal predispositions can lead to inconsistent decision-making. Furthermore, manual reviews are slow, often taking days or weeks to process a single application. In a market where customers expect "instant" everything, this lag time results in high abandonment rates. While some banks have moved to automated systems, many still rely on simple "decision trees" that cannot handle the complexity of modern financial lives, such as the rise of gig-economy workers or non-traditional income streams.

The "Black Box" and Legal Non-Compliance Standard machine learning models used in credit scoring—such as neural networks or gradient-boosted trees—are notoriously opaque. They might predict a high risk of default with 99% accuracy, but they cannot explain *why* in a way that a human can understand. This creates a massive problem for "Right to Explanation" laws. If an applicant is denied a loan, the bank is legally required to provide specific reasons. When the reason is "the model weights determined you were a risk," the bank faces potential lawsuits and regulatory fines. There is a desperate need for a system that combines the predictive power of AI with the explanatory capability of human language.

Inflexibility in the Face of 2026 Regulatory Changes The global financial landscape in 2026 has introduced strict new standards regarding debt-to-income (DTI) ratios, environmental impact scoring for loans, and digital identity verification. Traditional software architectures are "brittle"; they are built on hard-coded rules that are difficult to change. Every time a central bank updates its interest rate policy or a government changes lending laws, the bank's IT department must embark on a multi-month development cycle. This creates a "compliance gap" where the bank may unknowingly operate under outdated rules for months at a time, leading to significant legal exposure.

Algorithms & Logic

The intelligence of this system is founded on the concept of **Agentic Orchestration**, a sophisticated method of managing LLMs that allows them to perform specialized roles within a predefined workflow. Unlike a standard chatbot that answers questions linearly, this system uses a Directed Acyclic Graph (DAG) to control how data flows between different specialized agents. This section details the logic governing the first two agents in the sequence.

Agent 1: The Data Auditor (Integrity Logic)

The Data Auditor is the first line of defense. Its primary algorithm is based on **Consistency Mapping and Zero-Shot Validation**. When the user submits their data (Income, Debt, Employment), the Auditor doesn't just check if the fields are filled; it evaluates the "logical harmony" of the data.

- **Input Validation:** It identifies if the Annual Income is logically sufficient to cover the Total Debt.
- **Suspicion Detection:** If an applicant claims a high income but lists "Unemployed" as their status, the Auditor flags this as a "Logical Anomaly."
- **Flagging System:** The Auditor generates a JSON-formatted report containing binary flags (0 or 1). These flags are passed downstream to the Risk Orchestrator, where they act as "Penalty Multipliers" in the final mathematical model.

Agent 2: The RAG Policy Officer (Regulatory Logic)

The Policy Officer utilizes **Retrieval-Augmented Generation (RAG)** to ensure that the system is always compliant with the 2026 lending standards. This involves a three-step algorithmic process:

1. **Semantic Embedding:** The `lending_policy.txt` is broken down into text chunks. Each chunk is passed through an embedding model (like `text-embedding-3-small`) to create a 1536-dimensional vector representing its semantic meaning.
2. **Vector Retrieval:** When a loan application is processed, the Policy Officer generates a search query based on the applicant's profile (eg, "What are the specific requirements for a \$50,000 loan?"). It then performs a **Cosine Similarity** search against the vector database (**ChromaDB** or **FAISS**).

a. *The Formula:*
$$\cos(\theta) = \frac{A \cdot B}{|A| |B|}$$

3. **Contextual Reasoning:** The agent retrieves the top-3 most relevant policy

snippets. It then uses the LLM to read these snippets and compare them against the applicant's data. If the policy says "DTI must be below 40%" and the applicant is at 45%, the Policy Officer issues a "Non-Compliance" report, which is sent to the Orchestrator. This ensures that the final decision is grounded in "Ground Truth" documentation rather than the LLM's internal (and potentially outdated) training data.

Agent 3: The Risk Orchestrator (Synthesis & Synthesis Logic)

The final and most complex agent is the Risk Orchestrator. Its role is to act as the "Lead Underwriter," synthesizing the raw data, the Auditor's flags, the Policy Officer's compliance report, and the facts retrieved from external tools.

Tool-Augmented Retrieval (Function Calling)

The Orchestrator does not guess; it verifies. It uses **Function Calling** to execute Python tools that interact with deterministic systems:

- **fetch_credit_report(full_name):** This tool simulates an API call to a credit bureau like Equifax or TransUnion. It returns a structured object containing a numeric Credit Score (300-850) and a brief payment history summary.
- **bank_internal_check(full_name):** This tool queries the bank's internal database to check for existing accounts, average monthly balances, and any history of bounced checks or overdrafts.

The Decision Model (Mathematical Logic)

To ensure transparency and remove the "black box" nature of AI, the Orchestrator uses a weighted scoring model. While the LLM provides the qualitative reasoning, the final decision is validated by a mathematical formula. The **Probability of Success (\$P_s\$)** is calculated as follows:

$$P_s = w_1(C_{\text{score}}) + w_2\left(\frac{\text{Income}}{\text{Debt}}\right) - w_3(\text{Flags})$$

Where:

- **\$w_1\$ (Weight 1):** Typically 0.6. This places the highest priority on the Credit Bureau score.
- **\$w_2\$ (Weight 2):** Typically 0.3. This accounts for the applicant's liquidity (Debt-to-Income).
- **\$w_3\$ (Weight 3):** Typically 0.1 per flag. This penalizes the score based on suspicious data identified by Agent 1.

Final Classification Logic

The system applies a threshold τ (e.g., 0.75).

- If $P_s \geq \tau$ and the **Policy Officer** confirms zero "Critical Policy Violations," the application is marked as **APPROVED**.
- If $P_s < \tau$ or if the **Policy Officer** finds a violation (eg, DTI too high), the application is marked as **REJECTED**.

The Orchestrator then generates a final natural-language summary. It explains the decision by citing the specific math (using the LaTeX formula above) and the specific policy retrieved by Agent 2. This multi-layered approach ensures that the "reasoning" is not just a guess, but a verifiable calculation based on both qualitative policy and quantitative data.

Project Implementation & Technology Stack

The implementation of the **Multi-Agent Credit Risk & Loan Approval System** is designed with a "Production-First" mindset, emphasizing modularity, security, and a high-performance user interface. The architecture follows the **Model-View-Controller (MVC)** pattern, where the agents serve as the controllers, the data/tools as the model, and Streamlit as the view.

The Core Tech Stack

- **Language & Framework: Python 3.10+** is the backbone of the system, chosen for its extensive libraries in AI and data science.
- **LLM Engine:** The system utilizes **OpenAI GPT-4o** or **Anthropic Claude-3.5-Sonnet**. These models are selected for their high "Reasoning Density"—the ability to follow multi-step instructions and accurately call external functions without hallucinating.
- **Agent Orchestration: LangGraph** (part of the LangChain ecosystem) is used to manage the stateful, multi-actor flow. It allows for the creation of cycles and conditional edges, meaning if the Auditor finds a mistake, the workflow can "loop back" to ask for clarification.
- **Vector Storage: ChromaDB** is implemented as the local vector database. It is lightweight and allows for the storage of the `lending_policy.txt` embeddings with fast query response times.
- **Frontend UI: Streamlit** provides a rapid way to build a professional "Financial Dashboard." We utilize custom CSS to enforce a "Dark Mode" aesthetic, which is the standard for modern fintech professional tools.

Modular Codebase Structure

The project is split into four primary modules to ensure clean code practices:

1. **main.py(The Entry Point):** Handles the Streamlit UI layout. It captures user input, displays the "Reasoning Trace" using `st.status` or `st.expander`, and renders the final LaTeX math and approval badges.
2. **agents.py(The Brains):** Defines the "System Prompts" for each agent. This is where the persona and specific duties of the Data Auditor, Policy Officer, and Risk Orchestrator are coded.
3. **tools.py(The Hands):** Contains the Python functions that simulate API calls. This is where `fetch_credit_report` and `bank_internal_check` are defined using Python logic.

4. **vector_store.py(The Memory):** Handles the ingestion of `lending_policy.txt`. It reads the file, chunks the text, creates embeddings, and saves them to the ChromaDB collection.

Security and Key Management

Security is paramount in financial applications. We use **python-dotenv** to manage API keys. This ensures that sensitive credentials for OpenAI or Anthropic are never hardcoded and are instead loaded from a hidden `.envfile`. Furthermore, the "Reasoning Trace" is sanitized to ensure that while the *logic* is shown, sensitive user data is handled according to 2026 privacy standards.

Conclusion and Applications

The **Multi-Agent Credit Risk & Loan Approval System** represents a transformative approach to financial decision-making. By decomposing the monolithic task of "loan approval" into a collaborative effort between specialized AI agents, we have created a system that mimics human expertise while operating with machine-like speed and precision. The integration of **RAG** ensures that the system is never "out of date," as it can ingest new regulatory documents in seconds. Most importantly, the move towards **Explainable AI (XAI)** — demonstrated by the "Reasoning Trace" and the LaTeX-based mathematical justification—ensures that every decision is transparent, auditable, and legally defensible. This project proves that AI can be a "White Box" solution that builds trust between financial institutions and their customers.

Real-World Applications

The versatility of this agentic architecture allows it to be deployed across various sectors of the financial industry:

1. **Consumer Banking:** Automating the issuance of credit cards and personal loans. By providing instant decisions with clear explanations, banks can significantly improve customer satisfaction and conversion rates.
2. **SME and Business Lending:** Evaluating small business loans often requires checking complex internal policies and external credit scores simultaneously. This system can handle the multi-dimensional analysis required for business underwriting.
3. **Regulatory Compliance & Auditing:** The system can be used as a "Shadow Underwriter" to audit human decisions. If a human officer approves a loan that the AI agent flags as a "Policy Violation," it creates an immediate opportunity for risk mitigation.
4. **FinTech Platforms:** For "Buy Now, Pay Later" (BNPL) providers or Neo-banks, this system offers a low-cost, high-speed alternative to hiring massive teams of human underwriters.
5. **Educational Training:** The "Reasoning Trace" can serve as an educational tool for training new credit officers, showing them exactly how to interpret a 2026 lending policy against a real-world applicant's profile.

As we move further into 2026, the ability to combine **Generative AI** with **Structured Data** and **Deterministic Tools** will be the hallmark of successful financial institutions. This project serves as a foundational blueprint for that future, delivering a system that is as intelligent as it is transparent.